Системный язык программирования с зависимыми типами

Соколов Павел

24 января 2024

Путь к линейным зависимым типам

- Categorical Semantics of Linear Logic (2009) — исследование категорной семантики линейной логики
- Syntax and Semantics of Linear Dependent Types (2014) — нет зависимости от линейных типов
- I Got Plenty o’ Nuttin’ (2016) и другие QTT
- A Bunched Homotopy Type Theory for Synthetic Stable Homotopy Theory (2022) — зависимость через “marked” вхождения в тип, аналог модальности ! в линейной логике

The Symmetry Proof We Want

Proposition

$\mathsf{sym} : A \otimes B \simeq B \otimes A$

Proof.

To define $\mathsf{sym} : A \otimes B \to B \otimes A$, suppose we have $p : A \otimes B$. Then $\otimes$-induction allows us to assume $p \equiv x \otimes y$, and we have $y \otimes x$.

$$\mathsf{sym} :\equiv \lambda p.\mathsf{let}\ x \otimes y = p\ \mathsf{in}\ y \otimes x$$

Then to prove $\prod_{(p:A\otimes B)} \mathsf{sym}(\mathsf{sym}(p)) = p$, use $\otimes$-induction again: the goal reduces to $x \otimes y = x \otimes y$ for which we have reflexivity.

$$\mathsf{inv} :\equiv \lambda p.\mathsf{let}\ x \otimes y = p\ \mathsf{in}\ \mathsf{refl}_{x\otimes y}$$

$\square$

Colourful Variables

We need to prevent terms like $\lambda x.x \otimes x : A \to A \otimes A$, so variable use needs to be restricted somehow.

- Every variable x has a *colour* $\mathfrak{c}$.
- The relationships between colours are collected in a *palette*.

Palettes Φ are constructed by

$$1 \qquad \Phi_1 \otimes \Phi_2 \qquad \Phi_1, \Phi_2 \qquad \mathfrak{c} \qquad \mathfrak{c} \prec \Phi$$

Typical palettes:

$$\mathfrak{p} \prec \mathfrak{r} \otimes \mathfrak{b} \qquad \mathfrak{w} \prec (\mathfrak{p} \prec \mathfrak{r} \otimes \mathfrak{b}) \otimes \mathfrak{y} \qquad \mathfrak{p} \prec (\mathfrak{r} \otimes \mathfrak{b}, \mathfrak{r}' \otimes \mathfrak{b}')$$

(Similar to 'bunched' type theory P. W. O'Hearn and Pym 1999; P. O'Hearn 2003)

Using Colourful Variables

Building a term, we need to keep track of the current 'top colour'. Suppose the palette is $\mathfrak{p} \prec \mathfrak{r} \otimes \mathfrak{b}$, and we have variables

$$x^{\mathfrak{r}} : A, y^{\mathfrak{b}} : B, z^{\mathfrak{p}} : C.$$

- The top colour here is $\mathfrak{p}$.
- The only variable that can be used currently is $z : C$. (Using x here would correspond to a projection from one side of a tensor.)
- Ordinary type formers bind variables with the current top colour:

$$\sum_{(x:A)} B(x) \qquad \prod_{(x:A)} B(x) \qquad (\lambda x.b)$$

$$\mathrm{ind}_{+}(z.C, x.c_1, y.c_2, p) \qquad \mathrm{ind}_{=}(x.x'.p.C, x.c, p)$$

- The rules for $\otimes$ will grant us access to the other variables.

Rules for $\otimes$, Take 1

Let $\mathfrak{p}$ be the top colour.

- **Formation:** For closed* $A : \mathcal{U}$ and $B : \mathcal{U}$, there is a type $A \otimes B : \mathcal{U}$.
- **Introduction:** In palette* $\mathfrak{p} \prec \mathfrak{r} \otimes \mathfrak{b}$, for any terms $a : A$ with colour $\mathfrak{r}$ and $b : B$ with colour $\mathfrak{b}$, there is a term

$$a \ {}_{\mathfrak{r}}\otimes_{\mathfrak{b}} \ b : A \otimes B$$

- **Elimination:** Any term $p : A \otimes B$ may be assumed to be of the form $x \ {}_{\mathfrak{r}}\otimes_{\mathfrak{b}} \ y$ for some variables $x^{\mathfrak{r}} : A, y^{\mathfrak{b}} : B$ with $\mathfrak{p} \prec \mathfrak{r} \otimes \mathfrak{b}$, in a term $c : C[x \ {}_{\mathfrak{r}}\otimes_{\mathfrak{b}} \ y/z]$.

$$(\text{let } x \ {}_{\mathfrak{r}}\otimes_{\mathfrak{b}} \ y = p \text{ in } c) : C[p/z]$$

- **Computation:** If the term really is of the form $a \ {}_{\mathfrak{r}'}\otimes_{\mathfrak{b}'} \ b$, then

$$(\text{let } x \ {}_{\mathfrak{r}}\otimes_{\mathfrak{b}} \ y = a \ {}_{\mathfrak{r}'}\otimes_{\mathfrak{b}'} \ b \text{ in } c) \equiv c[\mathfrak{r}'/\mathfrak{r} \otimes \mathfrak{b}'/\mathfrak{b} \mid a/x, b/y]$$

Eg: Symmetry

Proposition

There is a function sym : $A \otimes B \to B \otimes A$

Proof.

Suppose have $p : A \otimes B$. Then $\otimes$-induction on p gives $x^{\mathfrak{r}} : A$ and $y^{\mathfrak{b}} : B$, where $\mathfrak{p} \prec \mathfrak{r} \otimes \mathfrak{b}$.

We need to form a purple term of $B \otimes A$, so 'split $\mathfrak{p}$ into $\mathfrak{b}$ and $\mathfrak{r}$'. Then we can form $y \,{}_{\mathfrak{b}}\!\otimes_{\mathfrak{r}}\, x : B \otimes A$.

$$\mathsf{sym} :\equiv \lambda p.\mathsf{let}\ x \,{}_{\mathfrak{r}}\!\otimes_{\mathfrak{b}}\, y = p\ \mathsf{in}\ y \,{}_{\mathfrak{b}}\!\otimes_{\mathfrak{r}}\, x$$

$\square$

But we don't have $\mathfrak{p} \prec \mathfrak{b} \otimes \mathfrak{r}$ literally, we need to build in the symmetric monoidal structure.

Palette Splits

Need a more general judgement for when the palette linearly splits into two monoidally combined pieces: $\Phi \vdash \vec{\mathfrak{r}} \mid \vec{\mathfrak{b}}$ split

Symmetry: In palette $\mathfrak{p} \prec \mathfrak{r} \otimes \mathfrak{b}$,

$$\mathfrak{b} \mid \mathfrak{r} \text{ split}$$

Associativity: In palette $\mathfrak{w} \prec (\mathfrak{p} \prec \mathfrak{r} \otimes \mathfrak{b}) \otimes \mathfrak{y}$,

$$\mathfrak{r} \mid (\mathfrak{b} \otimes \mathfrak{y}) \text{ split}$$

Cartesian weakening: In palette $\mathfrak{p} \prec (\mathfrak{r} \otimes \mathfrak{b}, \mathfrak{r}' \otimes \mathfrak{b}')$,

$$\mathfrak{r}' \mid \mathfrak{b}' \text{ split}$$

Rules for $\otimes$, Take 2

Let $\mathfrak{p}$ be the top colour.

- **Formation:** For closed* $A : \mathcal{U}$ and $B : \mathcal{U}$, there is a type $A \otimes B : \mathcal{U}$.
- **Introduction:** For any palette split $\vec{\mathfrak{r}} \mid \vec{\mathfrak{b}}$ and terms $a : A$ with colour $\vec{\mathfrak{r}}$ and $b : B$ with colour $\vec{\mathfrak{b}}$, there is a term

$$a \,{}_{\vec{\mathfrak{r}}}\!\otimes_{\vec{\mathfrak{b}}}\, b : A \otimes B$$

- **Elimination:** Any term $p : A \otimes B$ may be assumed to be of the form $x \,{}_{\mathfrak{r}}\!\otimes_{\mathfrak{b}}\, y$ for some variables $x^{\mathfrak{r}} : A, y^{\mathfrak{b}} : B$ with $\mathfrak{p} \prec \mathfrak{r} \otimes \mathfrak{b}$ in a term $c : C[x \,{}_{\mathfrak{r}}\!\otimes_{\mathfrak{b}}\, y/z]$.

$$(\text{let } x \,{}_{\mathfrak{r}}\!\otimes_{\mathfrak{b}}\, y = p \text{ in } c) : C[p/z]$$

- **Computation:** If the term really is of the form $a \,{}_{\vec{\mathfrak{r}'}}\!\otimes_{\vec{\mathfrak{b}'}}\, b$, then

$$(\text{let } x \,{}_{\mathfrak{r}}\!\otimes_{\mathfrak{b}}\, y = a \,{}_{\vec{\mathfrak{r}'}}\!\otimes_{\vec{\mathfrak{b}'}}\, b \text{ in } c) \equiv c[\vec{\mathfrak{r}'}/\mathfrak{r} \otimes \vec{\mathfrak{b}'}/\mathfrak{b} \mid a/x, b/y]$$

Eg: Uniqueness principle for $\otimes$

Proposition

If $C : A \otimes B \to \mathcal{U}$ is a type family and $f : \prod_{(p:A\otimes B)} C(p)$, then for any $p : A \otimes B$ we have

$$(\text{let } x \otimes y = p \text{ in } f(x \otimes y)) = f(p)$$

Proof.

By $\otimes$-induction we may assume $p \equiv x' \otimes y'$. Our goal is now

$$(\text{let } x \otimes y = x' \otimes y' \text{ in } f(x \otimes y)) = f(x' \otimes y')$$

Which by computation reduces to $f(x' \otimes y') = f(x' \otimes y')$, for which we have reflexivity. $\square$

(Cannot state this in indexed type or quantitative type theories)

Dependency in $\otimes$

From last time:

- Any assumption $x : A$ can be used 'marked': $\underline{x} : \underline{A}$.
- A $\underline{x}$ usage ignores the 'linear aspect' of x.
- A term a is *dull* if all free variables in a are marked.

Then we can allow the following dependency in $\otimes$:

- If $A : \mathcal{U}$ and $B : \mathcal{U}$ are *dull* types then $A \otimes B : \mathcal{U}$.
- If $A : \mathcal{U}$ is a dull type and B is a dull type assuming $x : A$, then $\bigotimes_{(\underline{x}:A)} B : \mathcal{U}$.

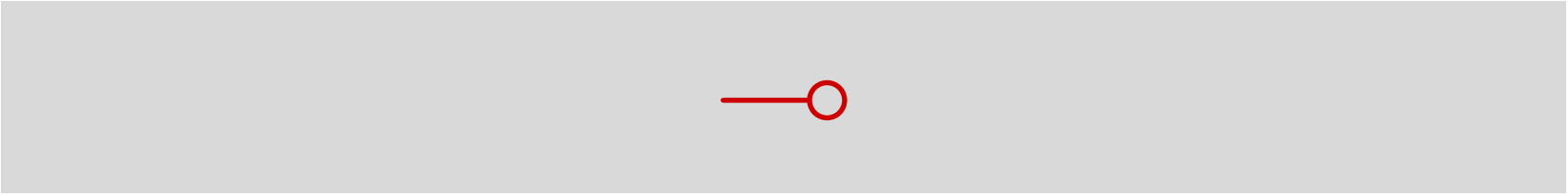
Eg. Associativity

Like dependent associativity of $\times$,

$$\begin{aligned}\mathsf{assoc} : &\left(\textstyle\sum_{(x:A)}\sum_{(y:B(x))} C(x)(y)\right)\\ \simeq &\left(\textstyle\sum_{(v:\sum_{(x:A)} B(x))} C(\mathsf{pr}_1 v)(\mathsf{pr}_2 v)\right)\end{aligned}$$

There is dependent associativity of $\otimes$:

$$\begin{aligned}\mathsf{assoc} : &\left(\textstyle\bigotimes_{(\underline{x}:A)}\bigotimes_{(\underline{y}:B(\underline{x}))} C(\underline{x})(\underline{y})\right)\\ \simeq &\left(\textstyle\bigotimes_{(\underline{v}:\bigotimes_{(\underline{x}:A)} B(\underline{x}))} \mathsf{let}\ x \otimes y = \underline{v}\ \mathsf{in}\ C(\underline{x})(\underline{y})\right)\end{aligned}$$



Hom

$$\frac{\Gamma \times A \vdash B}{\Gamma \vdash A \to B} \qquad \frac{\Gamma \otimes A \vdash B}{\Gamma \vdash A \multimap B}$$

Hom

$$\frac{\Gamma \times (x : A) \vdash b : B}{\Gamma \vdash \lambda x.b : \prod_{(x:A)} B} \qquad \frac{\Gamma \otimes (y : A) \vdash b : B}{\Gamma \vdash \partial y.b : \textcircled{\prod}_{(y:A)} B}$$

Hom

$$\frac{\mathfrak{r} \mid \Gamma, x^{\mathfrak{r}} : A \vdash b : B}{\mathfrak{r} \mid \Gamma \vdash \lambda x.b : \prod_{(x:A)} B} \qquad \frac{\mathfrak{p} \prec \mathfrak{r} \otimes \mathfrak{b} \mid \Gamma, y^{\mathfrak{b}} : A \vdash b : B}{\mathfrak{r} \mid \Gamma \vdash \partial y.b : \textcircled{\prod}_{(y^{\mathfrak{b}}:A)} B}$$

Типы указателей, типы допуска

Аналогично ATS и GhostCell (2021), за адресацию и за владение памятью отвечают два разных типа: тип ссылок $\&A$ и, для любых $r : \&A$ и $n : \mathrm{nat}$, допуск $[r; n)$.
Некоторые свойства:

- $\&A$ не является линейным;
- над $\&A$ можно производить арифметику на указателях;
- $[r; n)$ является линейным;
- $[r; n)$ можно разделять (на $[r; m)$ и $[r + m; n)$) и соединять обратно;
- Чтение и запись по $p : \&A$ можно производить только при наличии $[r; n)$ такого, что $r : \&A$ и $r \leq p < r + n$.

Ручное управление памятью

На основе типов данных, введённых ранее, можно ввести следующие операции:

$$\mathrm{malloc}_A : (n : \mathrm{nat}) \multimap \mathrm{ENOMEM} \oplus (r : \&A) \otimes [r; n)$$

$$\mathrm{realloc}_{A,r,n} : (m : \mathrm{nat}) \otimes [r; n) \multimap [r; n) \oplus (r' : \&A) \otimes [r'; m)$$

$$\mathrm{free}_{A,r,n} : [r; n) \multimap 1$$

Благодаря выразительности базовой системы типов соотношения, которым должна удовлетворять модель такой сигнатуры, можно выписать внутри теории. Возможные вариации операционных моделей с состоянием очевидны.

Задача 1: эффективная компиляция лямбда-выражений

1. Стандартное решение в C++, Rust: анонимный тип данных, сохраняющий контекст, с переопределённой операцией `()`.
   - Недостаток: нет одного типа функции $(x : A) \to B(x)$, есть семейство типов, объединённых одним концептом (C++) / трейтом (Rust).
2. Есть boxed вариант, в котором контекст хранится в куче, так что “лямбда” представляется в виде пары указателей — на данные и на функцию. Недостатки:
   - Потеря производительности по сравнению с первым вариантом;
   - Решение полагается на работу с кучей.
3. Заметим, что куча нужна только для того, чтобы обойти проблему с контекстами разной длины. Для решения на стеке можно дополнительно хранить в структуре её размер.

Задача 2: (ко-)рекурсивные типы данных (I)

- Аналогично задаче с лямбдами, представление индуктивных типов данных в памяти вынуждено использовать кучу; здесь обойти это ограничение не представляется возможным.
- Кроме этого, в системном программировании повсеместно используются не завершающиеся процедуры и явно циклические типы данных (например, двусвязный список). Для их моделирования в языках с зависимыми типами используются ко-данные и ко-рекурсия.

Задача 2: (ко-)рекурсивные типы данных (II)

- Для аккуратного анализа работы с ко-данными используется guarded recursion, см. Productive Coprogramming with Guarded Recursion (2013).
- Предложение: для представления ко-данных использовать типы вида $\nu A.F(\&A)$, для индуктивных типов данных — $\mu A.F((r : \mathrm{nat}) \otimes [r; 1))$.
- Индуктивные линейные типы — тема новейших исследований, см. A Calculus of Inductive Linear Constructions (2023).
- Направление дальнейших исследований — интеграция системы из статьи выше в нашу, исследование эргономики и её повышение.